

Detailed Look at Meteorological Station Out-of-Range Data

July 6, 2026

Contents

1	Introduction	3
2	Summary of Air Temperature Quality Control Results	3
3	CB04	4
3.1	Overview	4
3.2	Diagnostic Figures	4
4	FD03	5
4.1	Overview	5
4.2	Diagnostic Figures	5
5	WC05	7
5.1	Overview	7
5.2	Diagnostic Figures	7
6	Overall Correlation of Met Stations Over Time	8
6.1	Objective	8
6.2	Methodology	8
6.3	Python Script	8
6.4	Results and Interpretation	10
7	Correlation for Met Stations Within Two Weeks of Out-of-Range Data	12
7.1	Objective	12
7.2	Methodology	12
7.3	Python Script	12
7.4	Results and Interpretation	15
8	Discussion	15
9	Conclusions	15

1 Introduction

Meteorological station datasets were screened for missing values and observations falling outside established quality-control limits. Stations exhibiting out-of-range air temperature observations were investigated using graphical diagnostics to identify periods of anomalous behavior, potential sensor failures, and opportunities for data recovery through donor-station gap filling.

The purpose of this report is to document the identified air temperature anomalies, provide visual inspection of the affected stations, and summarize correlation analyses used to identify suitable donor stations for replacing invalid observations.

Stations with notable air temperature quality-control concerns were selected for detailed review. For each station, diagnostic figures are presented and discussed. Following the station-level analyses, correlation-based donor station selection methods are described and documented using the Python workflows developed for this project.

2 Summary of Air Temperature Quality Control Results

Table 1 summarizes stations containing air temperature observations that were flagged as out-of-range during quality-control screening.

Table 1: Air temperature out-of-range observations identified during quality-control screening.

Station	Out-of-Range Tair Observations
CB04	15,311
FD03	7,540
WC05	100
CB06	0
FD02	0

The most substantial issues occurred at CB04 and FD03, where long periods of anomalous air temperature observations were identified. WC05 contained a relatively small number of problematic observations, while CB06 and FD02 were retained in subsequent analyses because of their potential value as donor stations during gap-filling efforts.

3 CB04

3.1 Overview

CB04 exhibited the largest concentration of out-of-range air temperature observations among all meteorological stations examined. A total of 15,311 air temperature observations were identified as invalid during the screening process.

Visual inspection was performed to determine:

- Whether anomalies occurred continuously or intermittently.
- Whether the station exhibited abrupt shifts or sensor drift.
- Whether nearby stations could potentially be used as donor stations.

3.2 Diagnostic Figures

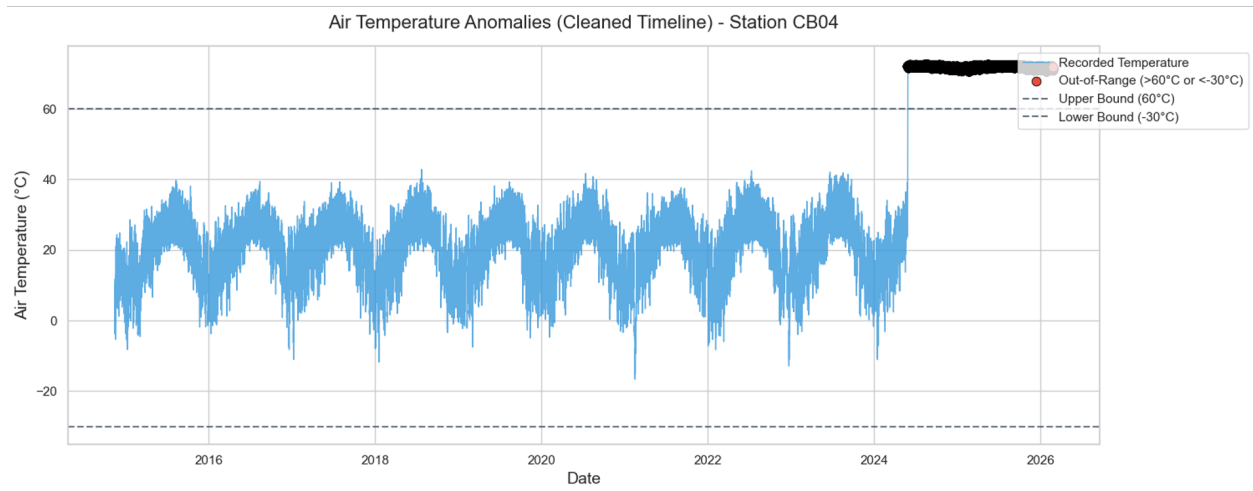


Figure 1: CB04 diagnostic figure 1.

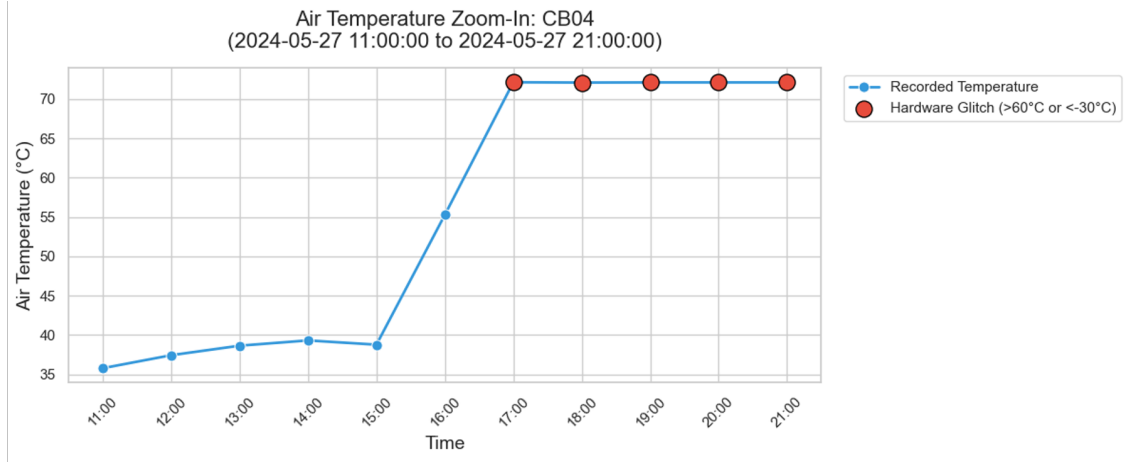


Figure 2: CB04 diagnostic figure 2.

4 FD03

4.1 Overview

FD03 contained the second-largest number of out-of-range air temperature observations. A total of 7,540 observations were identified during quality-control screening.

These diagnostic plots were used to assess:

- Timing and duration of anomalies.
- Potential sensor instability.
- Candidate donor stations for reconstruction of invalid data.

4.2 Diagnostic Figures

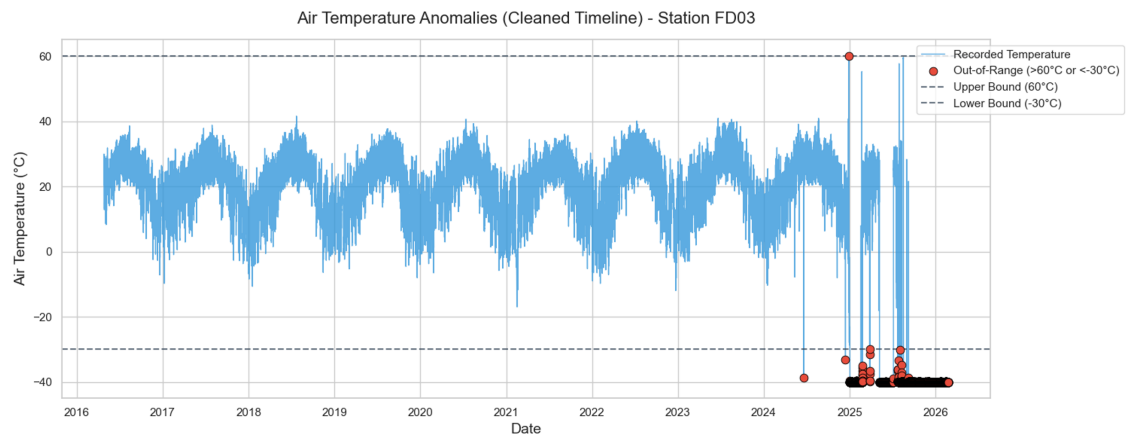


Figure 3: FD03 diagnostic figure 1.

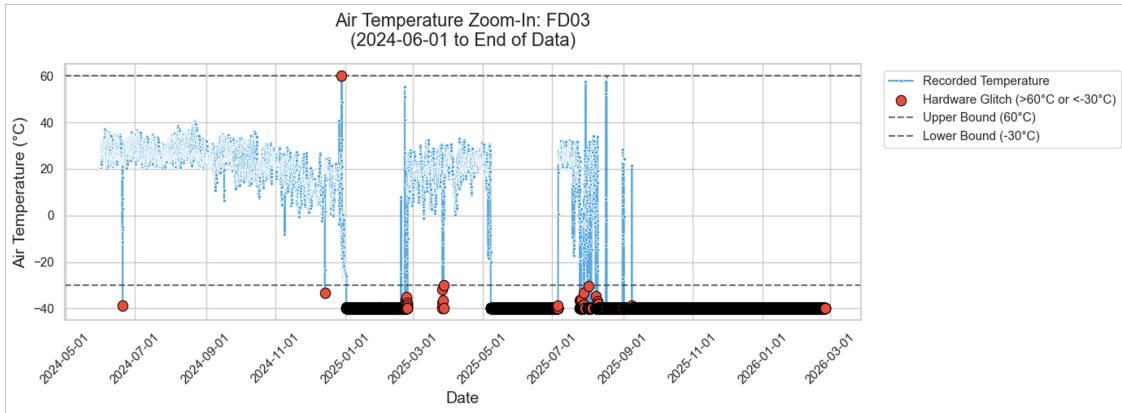


Figure 4: FD03 diagnostic figure 2.

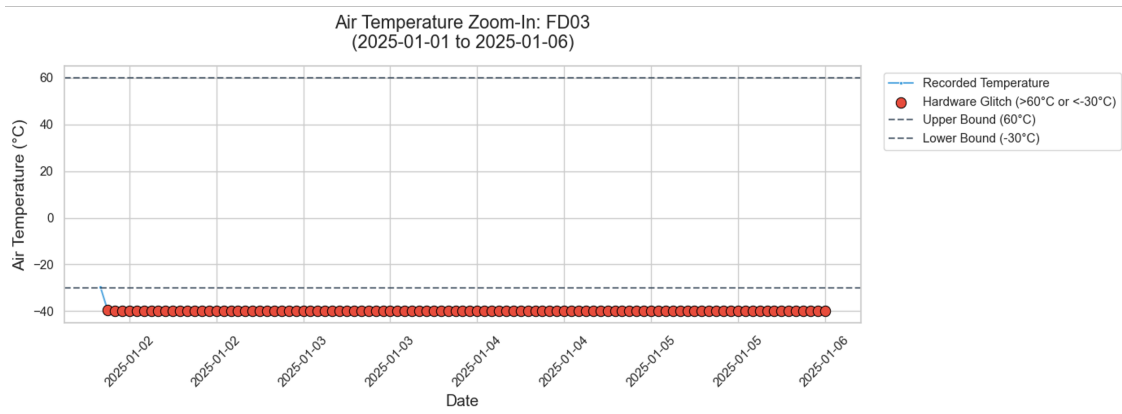


Figure 5: FD03 diagnostic figure 3.

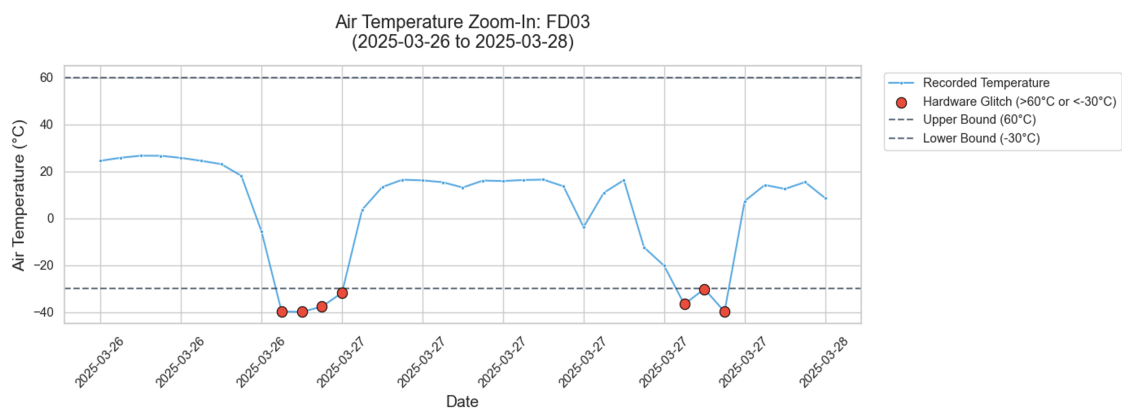


Figure 6: FD03 diagnostic figure 4.

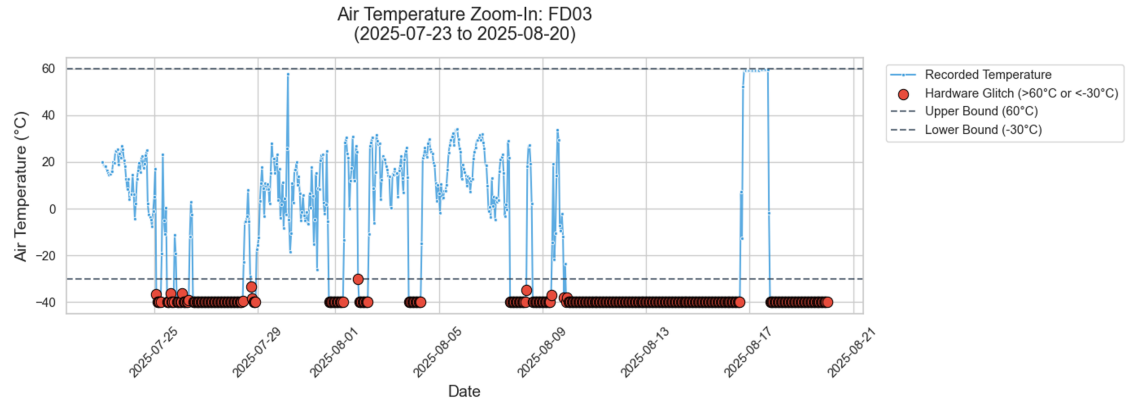


Figure 7: FD03 diagnostic figure 5.

5 WC05

5.1 Overview

WC05 contained a relatively small number of out-of-range air temperature observations. Although only 100 observations were identified as invalid, visual inspection was conducted to determine whether those observations were isolated events or indicative of broader data-quality concerns.

5.2 Diagnostic Figures

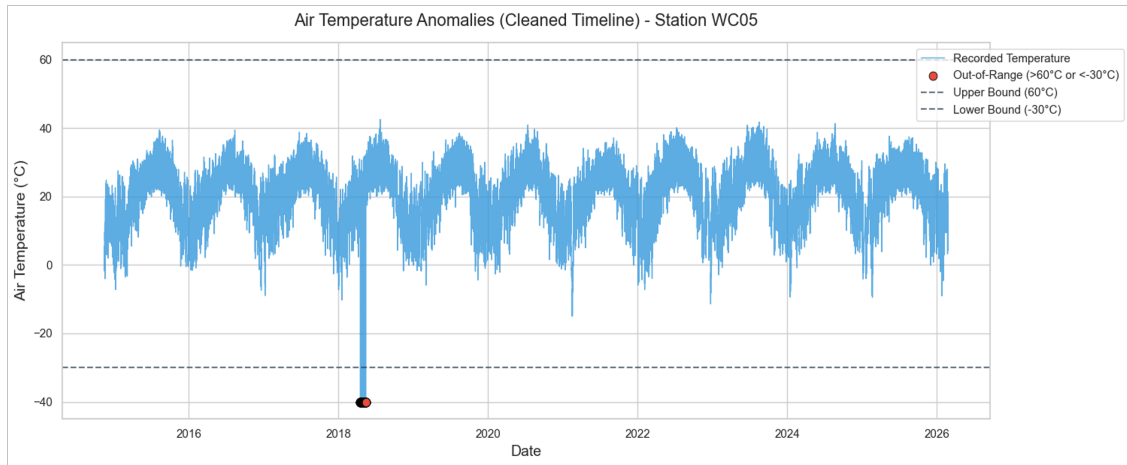


Figure 8: WC05 diagnostic figure 1.

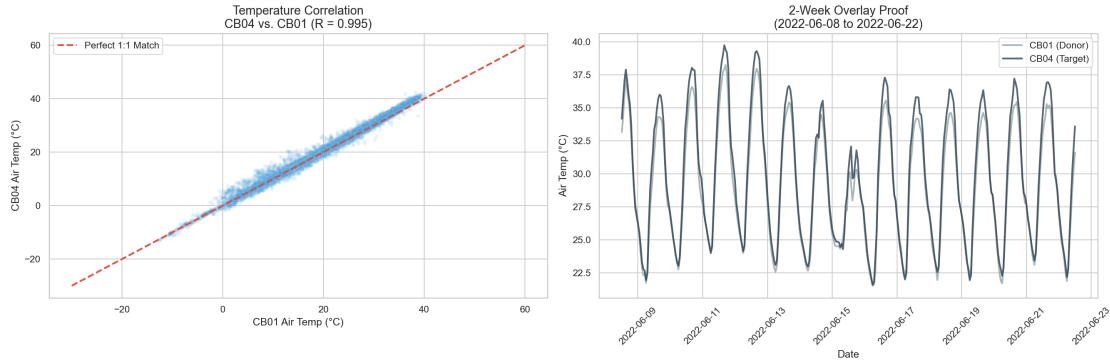


Figure 9: WC05 diagnostic figure 2.

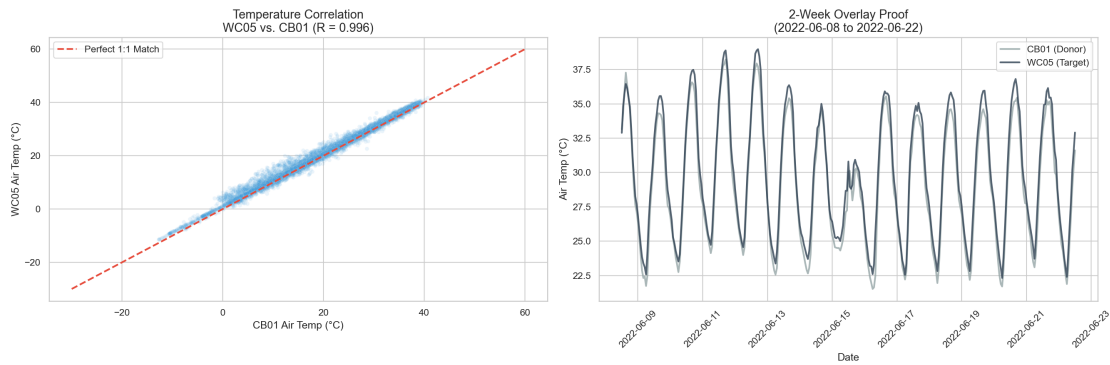


Figure 10: WC05 diagnostic figure 3.

6 Overall Correlation of Met Stations Over Time

6.1 Objective

The purpose of this analysis is to determine which meteorological stations exhibit the strongest long-term agreement in air temperature observations. The resulting correlation coefficients are used to identify donor stations that may be used to replace invalid or missing observations.

6.2 Methodology

For each target station, air temperature records are compared against all available stations using the full period of record. Correlation coefficients are calculated using all overlapping observations. The station with the highest correlation is considered the preferred donor station for long-term gap-filling applications.

This approach allows donor stations to be selected based on observed similarity rather than geographic proximity alone.

6.3 Python Script

Paste the complete contents of `air_temp_corr.py` below.

```
1 import pandas as pd
```

```

2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from pathlib import Path
5
6 # Set visualization style
7 sns.set_theme(style="whitegrid")
8
9 def find_best_donor_station(target_station, cleaned_met_dir="cleaned_met_data"):
10     """Finds and visualizes the best matching station to fill temperature gaps."""
11     met_dir = Path(cleaned_met_dir)
12
13     if not met_dir.exists():
14         print(f"Error: Could not find {met_dir}")
15         return
16
17     print("1. Loading all cleaned Met stations to build the Temperature Matrix...")
18
19     # 1. Build a Master DataFrame of just Tair
20     tair_data = {}
21     for file_path in met_dir.glob("*_met_cleaned.csv"):
22         station_id = file_path.name.replace("_met_cleaned.csv", "")
23         # Read the file, grab only the Tair column to save memory
24         df = pd.read_csv(file_path, parse_dates=['Date'], index_col='Date',
25                          usecols=['Date', 'Tair'])
26         tair_data[station_id] = df['Tair']
27
28     if target_station not in tair_data:
29         print(f"Error: Target station {target_station} not found in cleaned data.")
30         return
31
32     # Combine all series into one massive DataFrame
33     master_tair_df = pd.DataFrame(tair_data)
34
35     print("2. Calculating correlation matrix...")
36     # Calculate Pearson Correlation (ignoring NaNs automatically)
37     corr_matrix = master_tair_df.corr()
38
39     # 3. Find the best matches for our target station
40     target_correlations = corr_matrix[target_station].drop(target_station)
41     top_matches = target_correlations.sort_values(ascending=False).head(3)
42
43     best_match_station = top_matches.index[0]
44     best_match_score = top_matches.iloc[0]
45
46     print(f"\n--- Top 3 Donor Stations for {target_station} ---")
47     for stat, score in top_matches.items():
48         print(f"    {stat}: {score:.4f} correlation")
49
50     # --- VISUALIZATION ---
51     print(f"\n3. Generating visual comparison between {target_station} and {best_match_station}...")
52
53     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
54
55     # Plot 1: Scatter Plot of Target vs. Best Match
56     # A perfect match would be a straight 45-degree diagonal line
57     sns.scatterplot(x=master_tair_df[best_match_station], y=master_tair_df[

```



```

57         target_station],
58         ax=ax1, alpha=0.1, color='#3498db', s=20)
59
60     ax1.plot([-30, 60], [-30, 60], color='#e74c3c', linestyle='--', linewidth=2,
61             label='Perfect 1:1 Match')
62     ax1.set_title(f"Temperature Correlation\n{target_station} vs. {
63         best_match_station} (R = {best_match_score:.3f})", fontsize=14)
64     ax1.set_xlabel(f"{best_match_station} Air Temp ( C )", fontsize=12)
65     ax1.set_ylabel(f"{target_station} Air Temp ( C )", fontsize=12)
66     ax1.legend()
67
68     # Plot 2: Time-Series Overlay (Zooming in on a random 2-week period where both
69     # have data)
70     # Drop NaNs just to find a good overlapping time window
71     valid_overlap = master_tair_df[[target_station, best_match_station]].dropna()
72     if not valid_overlap.empty:
73         start_plot = valid_overlap.index[0]
74         end_plot = start_plot + pd.Timedelta(days=14)
75         subset = master_tair_df.loc[start_plot:end_plot]
76
77         sns.lineplot(data=subset, x=subset.index, y=best_match_station, ax=ax2,
78                     color='#95a5a6', linewidth=2, label=f'{best_match_station} (
79                         Donor)', alpha=0.8)
80         sns.lineplot(data=subset, x=subset.index, y=target_station, ax=ax2,
81                     color='#2c3e50', linewidth=2, label=f'{target_station} (
82                         Target)', alpha=0.8)
83
84         ax2.set_title(f"2-Week Overlay Proof\n({start_plot.strftime('%Y-%m-%d')}
85             to {end_plot.strftime('%Y-%m-%d')})", fontsize=14)
86         ax2.set_xlabel("Date", fontsize=12)
87         ax2.set_ylabel("Air Temp ( C )", fontsize=12)
88         ax2.legend()
89         plt.xticks(rotation=45)
90
91     plt.tight_layout()
92
93     # Create a folder for the graphs
94     output_dir = Path("visualizations")
95     output_dir.mkdir(exist_ok=True)
96
97     # Save the graph as an image file
98     plt.savefig(output_dir / f"{target_station}_correlation_plot.png", dpi=150)
99     plt.close() # Closes the plot so it doesn't take up memory
100
101     print(f" -> Graph saved to {output_dir.name} folder!")
102
103 # Run it for one of your stations!
104 find_best_donor_station('CB04')
105 find_best_donor_station('FD03')
106 find_best_donor_station('WC05')

```

6.4 Results and Interpretation

The resulting correlation coefficients provide a quantitative measure of station similarity over time. Stronger correlations indicate that the donor station is more likely to reproduce temperature variability at the target station and therefore represents a better candidate for replacing invalid observations. Below are the strongest correlated stations for each of CB04, FD03 and WD05:

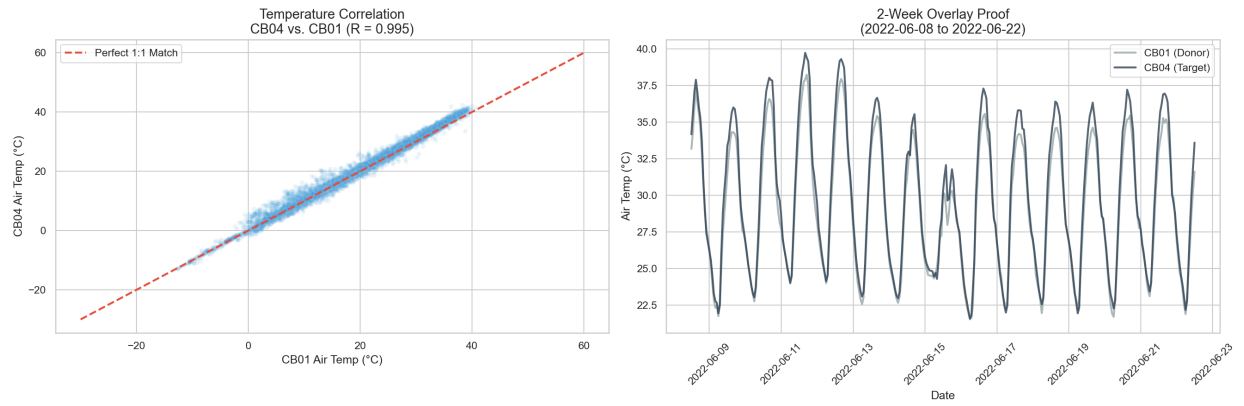


Figure 11: CB04 vs. CB01

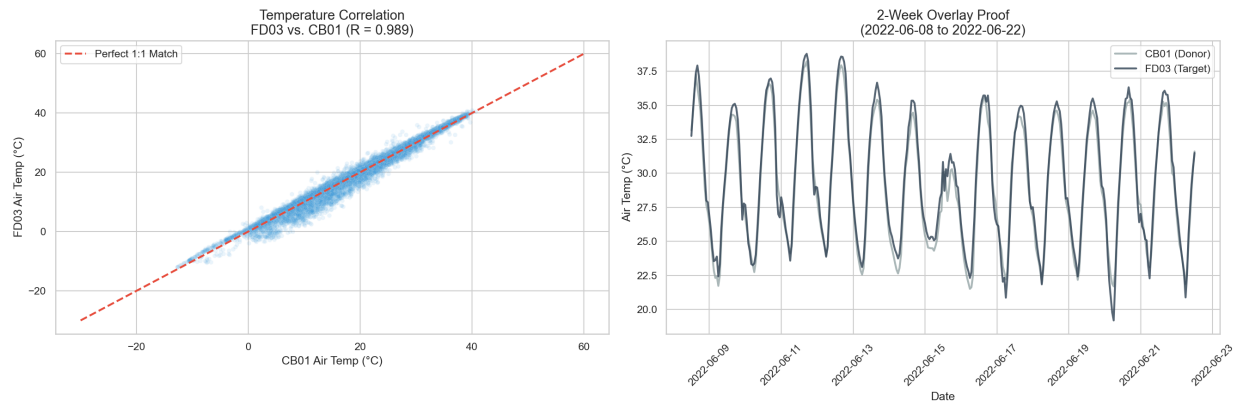


Figure 12: FD03 vs. CB04

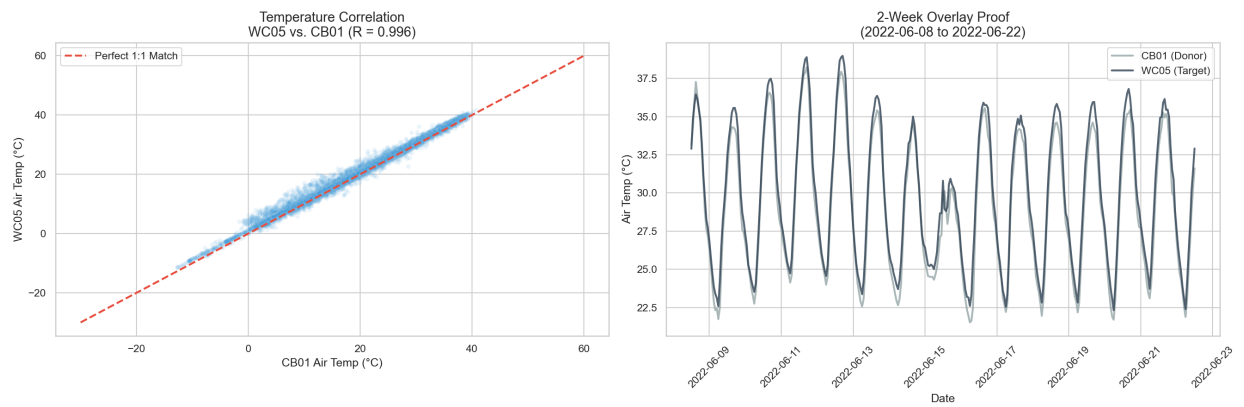


Figure 13: WC05 vs. CB01

7 Correlation for Met Stations Within Two Weeks of Out-of-Range Data

7.1 Objective

A station that performs well over the full period of record may not be the best donor during a specific out-of-range event. To address this possibility, a second correlation analysis was conducted using only observations occurring within two weeks before and two weeks after each out-of-range period.

7.2 Methodology

For each identified anomaly period:

1. A 14-day period preceding the anomaly is extracted.
2. A 14-day period following the anomaly is extracted.
3. The periods are combined into a local comparison window.
4. Correlations are recalculated among candidate donor stations.
5. The highest-correlated station is selected as the preferred donor for that event.

This approach better captures local weather conditions and short-term station behavior immediately surrounding the period requiring correction.

7.3 Python Script

Paste the complete contents of `air_temp_two_weeks.py` below.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from pathlib import Path
5
6 # Set visualization style
7 sns.set_theme(style="whitegrid")
8
9 def find_local_donor_station(target_station, gap_start, gap_end, feature='Tair',
10                             cleaned_dir="cleaned_met_data"):
11     """Finds best matching station based ONLY on the 14 days before and after a
12         specific gap and saves the plot."""
13     data_dir = Path(cleaned_dir)
14
15     if not data_dir.exists():
16         print(f"Error: Could not find {data_dir}")
17         return
18
19     # 1. Convert the gap strings to actual datetime objects
20     start_dt = pd.to_datetime(gap_start)
21     end_dt = pd.to_datetime(gap_end)
22
23     # 2. Define our 14-day comparison windows
24     pre_start = start_dt - pd.Timedelta(days=14)
25     post_end = end_dt + pd.Timedelta(days=14)
```

```

24
25 print(f"Analyzing local correlation for {target_station} ({feature})")
26 print(f"Gap Period: {start_dt} to {end_dt}")
27 print(f"Analyzing Window: {pre_start.date()} to {post_end.date()} (excluding
    the gap itself)\n")
28
29 # 3. Load all data for the specific feature to save RAM
30 feature_data = {}
31 for file_path in data_dir.glob(f"*_{data_dir.name.split('_')[1]}_cleaned.csv"):
32     station_id = file_path.name.split('_')[0]
33     df = pd.read_csv(file_path, parse_dates=['Date'], index_col='Date',
34                     usecols=['Date', feature])
35     feature_data[station_id] = df[feature]
36
37 if target_station not in feature_data:
38     print(f"Error: Target station {target_station} not found in {cleaned_dir}.")
39     return
40
41 master_df = pd.DataFrame(feature_data)
42
43 # 4. Filter the data to ONLY the Pre-Gap and Post-Gap windows
44 pre_mask = (master_df.index >= pre_start) & (master_df.index < start_dt)
45 post_mask = (master_df.index > end_dt) & (master_df.index <= post_end)
46
47 # Combine the valid windows
48 window_df = master_df[pre_mask | post_mask]
49
50 # 5. Calculate Correlation on this specific localized subset
51 corr_matrix = window_df.corr()
52 target_correlations = corr_matrix[target_station].drop(target_station)
53 top_matches = target_correlations.sort_values(ascending=False).head(3)
54
55 best_match_station = top_matches.index[0]
56 best_match_score = top_matches.iloc[0]
57
58 print(f"--- Top 3 Local Donor Stations ---")
59 for stat, score in top_matches.items():
60     print(f" {stat}: {score:.4f} correlation")
61
62 # --- VISUALIZATION ---
63 print(f"\nGenerating visual comparison between {target_station} and {
64     best_match_station}...")
65
66 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
67
68 # Plot 1: Scatter Plot of Target vs. Best Match (Using only the local window
69 data)
70 sns.scatterplot(x=window_df[best_match_station], y=window_df[target_station],
71                 ax=ax1, alpha=0.5, color='#3498db', s=30)
72
73 ax1.plot([-30, 60], [-30, 60], color='#e74c3c', linestyle='--', linewidth=2,
74          label='Perfect 1:1 Match')
75 ax1.set_title(f"Local Correlation (14 Days Pre/Post Gap)\n{target_station} vs.
76 {best_match_station} (R = {best_match_score:.3f})", fontsize=14)
77 ax1.set_xlabel(f"{best_match_station} {feature} ( C )", fontsize=12)
78 ax1.set_ylabel(f"{target_station} {feature} ( C )", fontsize=12)
79 ax1.legend()

```

```

75
76 # Plot 2: Time-Series Overlay of the Pre-Gap Window
77 pre_window_df = master_df.loc[pre_start:start_dt].dropna(subset=[
78     target_station, best_match_station])
79
80 if not pre_window_df.empty:
81     sns.lineplot(data=pre_window_df, x=pre_window_df.index, y=
82         best_match_station, ax=ax2,
83         color='#95a5a6', linewidth=2, label=f'{best_match_station} (
84             Donor)', alpha=0.8)
85     sns.lineplot(data=pre_window_df, x=pre_window_df.index, y=target_station,
86         ax=ax2,
87         color='#2c3e50', linewidth=2, label=f'{target_station} (
88             Target)', alpha=0.8)
89
90     ax2.set_title(f"Pre-Gap Behavior Overlay\n({pre_start.strftime('%Y-%m-%d')}
91         } to {start_dt.strftime('%Y-%m-%d')})", fontsize=14)
92     ax2.set_xlabel("Date", fontsize=12)
93     ax2.set_ylabel(f"{feature} ( C )", fontsize=12)
94     ax2.legend()
95     plt.xticks(rotation=45)
96
97 plt.tight_layout()
98
99 # --- SAVE THE PLOT ---
100 output_dir = Path("visualizations")
101 output_dir.mkdir(exist_ok=True)
102
103 # Format a clean filename using the station, feature, and date
104 safe_date = start_dt.strftime('%Y-%m-%d')
105 filename = f"{target_station}_{feature}_Local_Match_{safe_date}.png"
106
107 plt.savefig(output_dir / filename, dpi=150)
108 plt.close() # Closes the plot so it doesn't print to the notebook and saves
109             memory
110
111 print(f" -> Graph successfully saved to {output_dir.name}/{filename}!")
112
113 # --- Run it using a specific gap from your summary CSV ---
114 find_local_donor_station('CB04', '2024-05-27 17:00:00', '2026-02-24 15:00:00',
115     feature='Tair', cleaned_dir="cleaned_met_data")
116 find_local_donor_station('FD03', '2024-06-20 03:00:00', '2024-06-20 03:00:00',
117     feature='Tair', cleaned_dir="cleaned_met_data")
118 find_local_donor_station('FD03', '2025-01-01 21:00:00', '2025-02-18 15:00:00',
119     feature='Tair', cleaned_dir="cleaned_met_data")
120 find_local_donor_station('FD03', '2025-02-18 18:00:00', '2025-02-21 18:00:00',
121     feature='Tair', cleaned_dir="cleaned_met_data")
122 find_local_donor_station('FD03', '2025-05-08 03:00:00', '2025-07-06 01:00:00',
123     feature='Tair', cleaned_dir="cleaned_met_data")
124 find_local_donor_station('FD03', '2025-09-08 16:00:00', '2026-02-24 15:00:00',
125     feature='Tair', cleaned_dir="cleaned_met_data")
126 find_local_donor_station('WC05', '2018-04-29 08:00:00', '2018-04-29 18:00:00',
127     feature='Tair', cleaned_dir="cleaned_met_data")

```

7.4 Results and Interpretation

Local correlation analyses provide event-specific donor station recommendations. These correlations may differ substantially from full-period correlations because of seasonal shifts, changing weather patterns, and station-specific behavior. Consequently, localized analyses often provide the strongest basis for reconstructing air temperature data during anomalous periods.

8 Discussion

The review identified two stations, CB04 and FD03, as the primary sources of air temperature quality-control concerns. These stations contained large numbers of observations that failed screening criteria and therefore warranted detailed graphical inspection.

Correlation analyses were conducted using both long-term records and localized two-week windows surrounding anomaly periods. Together, these approaches provide a systematic framework for identifying suitable donor stations and improving confidence in reconstructed air temperature datasets.

The combination of visual diagnostics and quantitative donor-station selection procedures supports reproducible and defensible data-quality corrections for subsequent ecological and environmental analyses.

9 Conclusions

Air temperature quality-control screening identified substantial anomalies at CB04 and FD03, with smaller issues occurring at WC05. Diagnostic figures were reviewed to characterize the timing and persistence of these observations. To support data recovery efforts, donor-station selection analyses were performed using both full-period and event-specific correlations.

The methodologies documented in this report provide a framework for identifying appropriate replacement data and improving the quality of meteorological records used in downstream analyses.